

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Вычислительная Математика

Лабораторная работа №1

Вариант 19

Выполнил:
Чэнь Хаолинъ Р3216 407960
Преподаватель:
Наумова Надежда Александровна

г. Санкт-Петербург, 2026

Содержание

1	Цель работы	2
2	Задание	2
3	Блок-схема реализованного алгоритма	2
4	Реализация численного метода	6
5	Ссылка на GitHub с основной реализацией	9
6	Пример работы программы	9
7	Вывод	9

1. Цель работы

Изучить численные методы решения систем линейных алгебраических уравнений и реализовать один из них средствами программирования.

2. Задание

Метод Гаусса с выбором главного элемента по столбцам.

3. Блок-схема реализованного алгоритма

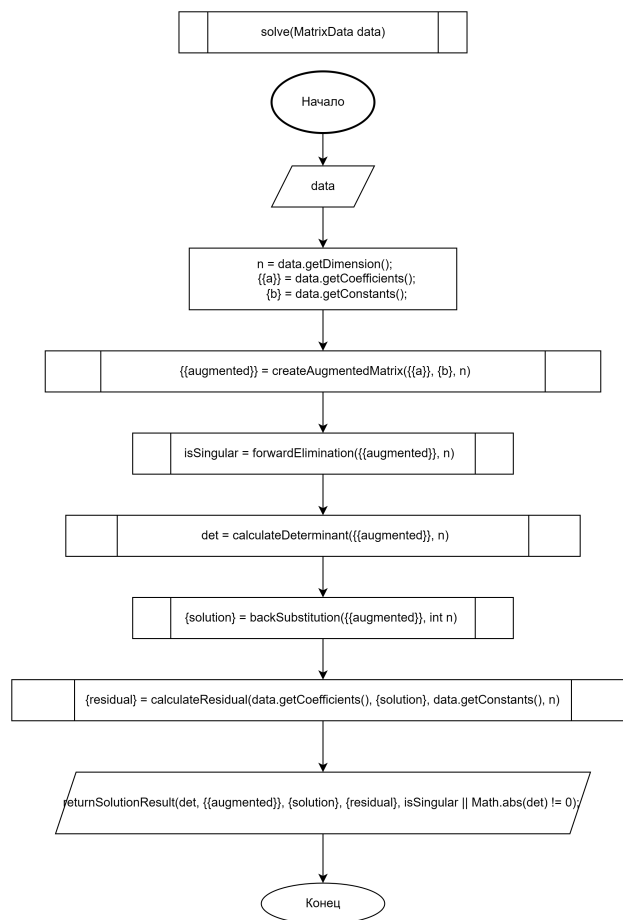


Рис. 1: Method solve

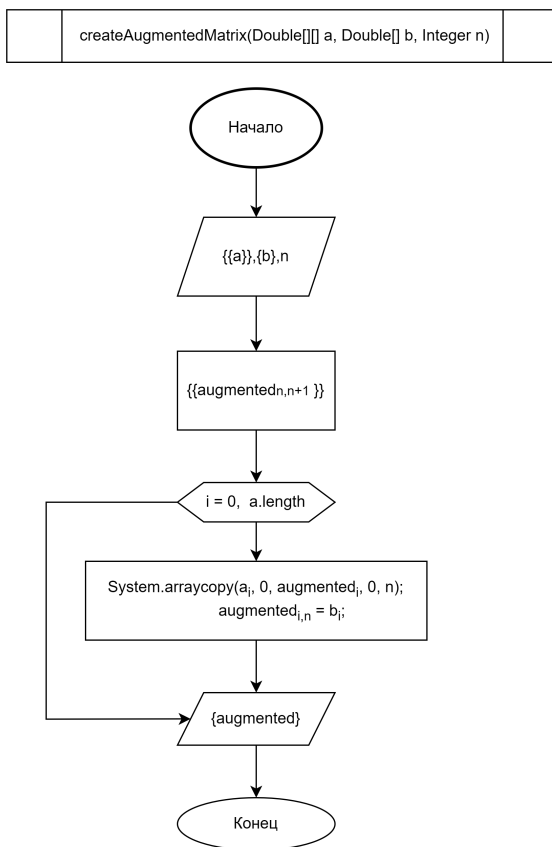


Рис. 2: Method createAugmentedMatrix

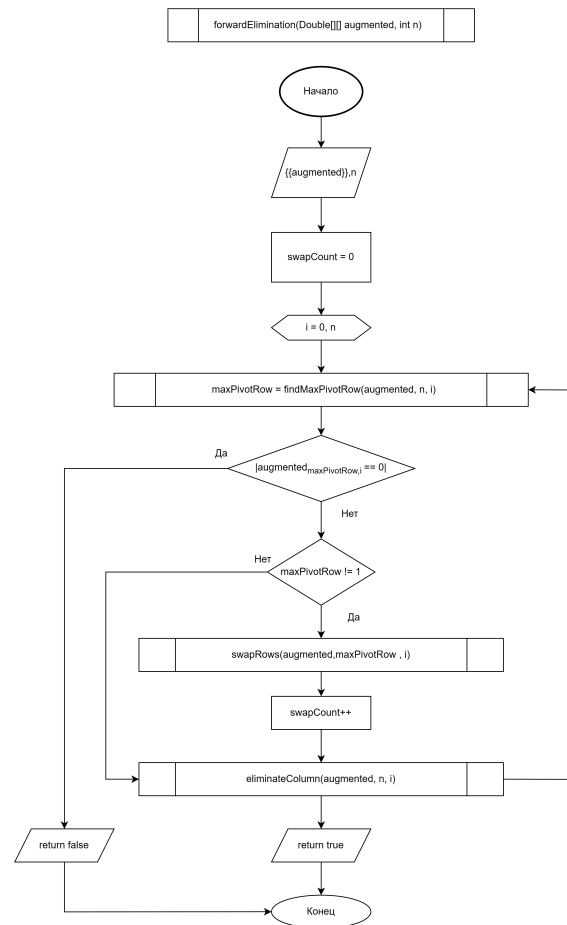


Рис. 3: Method forwardElimination

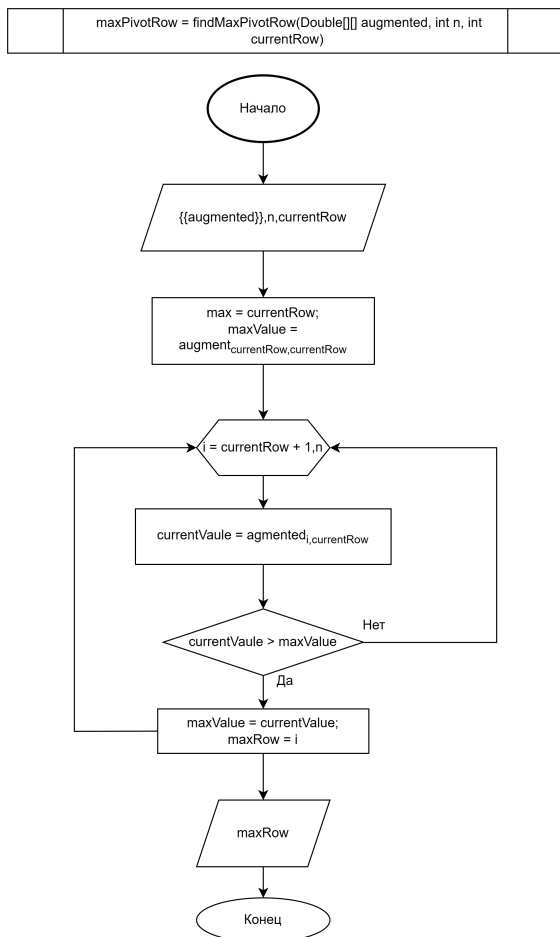


Рис. 4: Method findMaxPivotRow

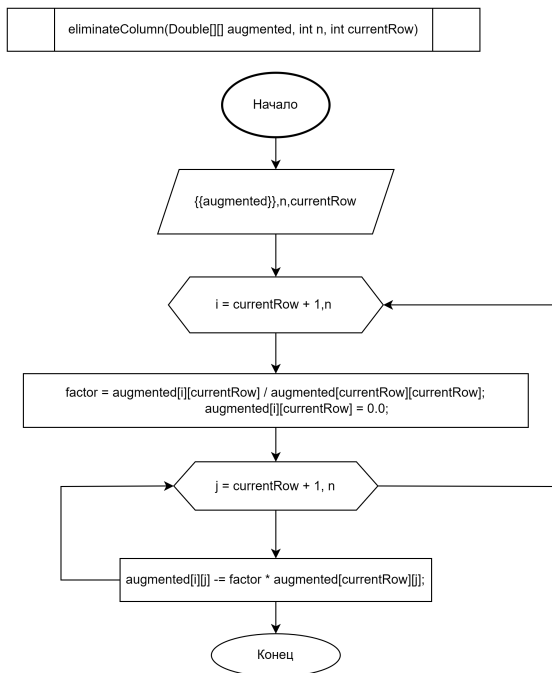


Рис. 6: Method eliminateColumn

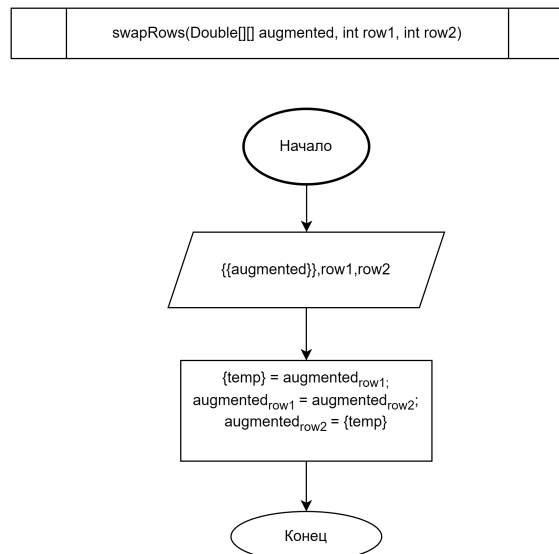


Рис. 5: Method swapRows

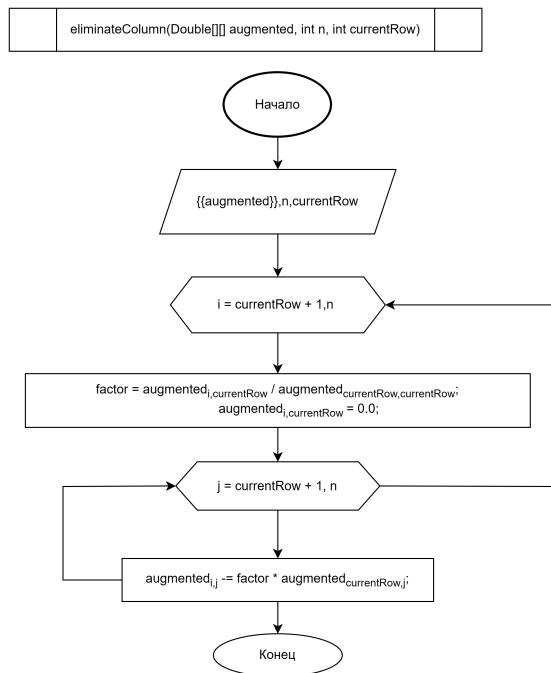


Рис. 7: Method calculateDeterminant

	backSubstitution(Double[][] upperAugmented, int n)	
--	--	--

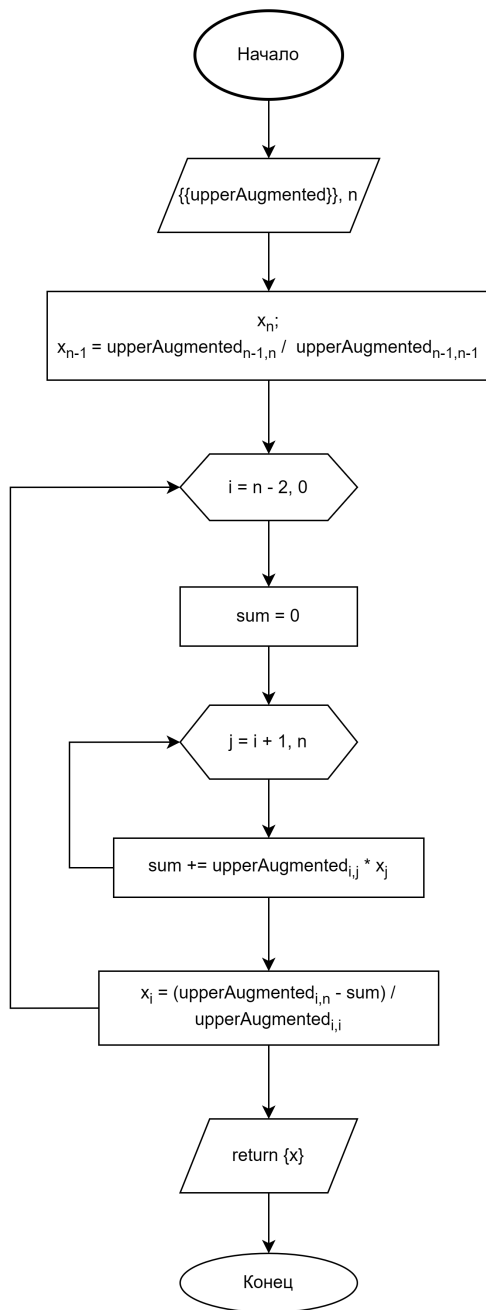


Рис. 8: Method backSubstitution

	calculateResidual(Double[][] originalA, Double[] solution, Double[] originalB, int n)	
--	---	--

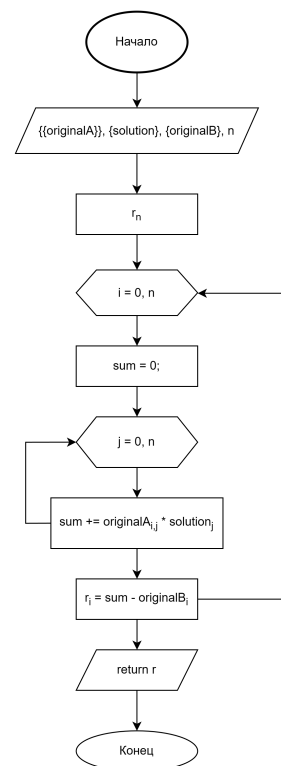


Рис. 9: Method calculateResidual

4. Реализация численного метода

```
1 package solver;
2
3 import model.MatrixData;
4 import model.SolutionResult;
5
6 public class GaussianEliminationSolver {
7     private int swapCount;
8
9     public SolutionResult solve(MatrixData data) {
10         int n = data.getDimension();
11         Double[][] a = data.getCoefficients();
12         Double[] b = data.getConstants();
13
14         Double[][] augmented = createAugmentedMatrix(a, b, n);
15
16         boolean isSingular = forwardElimination(augmented, n);
17
18         double det = calculateDeterminant(augmented, n);
19
20         Double[] solution = backSubstitution(augmented, n);
21
22         Double[] residual =
23             calculateResidual(data.getCoefficients(),
24                             solution, data.getConstants(), n);
25
26         return
27             new SolutionResult(det, augmented, solution,
28                             residual, isSingular || Math.abs(det) != 0);
29     }
30
31     private Double[][] createAugmentedMatrix(Double[][] a, Double[] b, Integer
32         n) {
33         Double[][] augmented = new Double[n][n + 1];
34
35         for (int i = 0; i < a.length; i++) {
36             System.arraycopy(a[i], 0, augmented[i], 0, n);
37             augmented[i][n] = b[i];
38         }
39
40         return augmented;
41     }
42
43     private boolean forwardElimination(Double[][] augmented, int n) {
44         swapCount = 0;
45
46         for (int i = 0; i < n; i++) {
47             int maxPivotRow = findMaxPivotRow(augmented, n, i);
48
49             if (Math.abs(augmented[maxPivotRow][i]) == 0) {
50                 return false;
51             }
52
53             if (maxPivotRow != i) {
54                 swapRows(augmented, maxPivotRow, i);
55                 swapCount++;
56             }
57         }
58     }
```

```

56         eliminateColumn(augmented, n, i);
57     }
58
59
60
61     return true;
62 }
63
64 private int findMaxPivotRow(Double[][] augmented, int n, int currentRow) {
65     int maxRow = currentRow;
66     double maxValue = Math.abs(augmented[currentRow][currentRow]);
67
68     for (int i = currentRow + 1; i < n; i++) {
69         double currentValue = Math.abs(augmented[i][currentRow]);
70         if (currentValue > maxValue) {
71             maxValue = currentValue;
72             maxRow = i;
73         }
74     }
75
76     return maxRow;
77 }
78
79 private void swapRows(Double[][] augmented, int row1, int row2) {
80     Double[] temp = augmented[row1];
81     augmented[row1] = augmented[row2];
82     augmented[row2] = temp;
83 }
84
85 private void eliminateColumn(Double[][] augmented, int n, int currentRow) {
86     for (int i = currentRow + 1; i < n; i++) {
87         Double factor = augmented[i][currentRow] /
88             augmented[currentRow][currentRow];
89         augmented[i][currentRow] = 0.0;
90         for (int j = currentRow + 1; j <= n; j++) {
91             augmented[i][j] -= factor * augmented[currentRow][j];
92         }
93     }
94 }
95
96 private Double[] backSubstitution(Double[][] upperAugmented, int n) {
97     Double[] x = new Double[n];
98
99     x[n - 1] = upperAugmented[n - 1][n] / upperAugmented[n - 1][n - 1];
100
101     //(x_{n-1}   x_1)
102     for (int i = n - 2; i >= 0; i--) {
103         double sum = 0;
104
105         //a_i(i+1)*x_(i+1) + ... + a_in*x_n
106         for (int j = i + 1; j < n; j++) {
107             sum += upperAugmented[i][j] * x[j];
108         }
109
110         //x_i = (b_i - sum) / a_ii
111         x[i] = (upperAugmented[i][n] - sum) / upperAugmented[i][i];
112     }

```



```

113     return x;
114 }
115
116 private Double[] calculateResidual(Double[][] originalA, Double[] solution,
117                                   Double[] originalB, int n) {
118     Double[] r = new Double[n];
119     for (int i = 0; i < n; i++) {
120         double sum = 0;
121         for (int j = 0; j < n; j++) {
122             sum += originalA[i][j] * solution[j];
123         }
124         r[i] = sum - originalB[i];
125     }
126     return r;
127 }
128
129 private double calculateDeterminant(Double[][] augmented, int n) {
130     double det = 1.0;
131     for (int i = 0; i < n; i++) {
132         det *= augmented[i][i];
133     }
134     return (swapCount % 2 == 0) ? det : -det;
135 }
136 }
137
138 }

```

5. Ссылка на GitHub с основной реализацией

[GitHub - Лабораторная работа №1](#)

6. Пример работы программы

```
=== Решение системы линейных уравнений методом Гаусса с выбором главного элемента ===
Выберите способ ввода:
1. Ввод с клавиатуры
2. Чтение из файла
3. Генерация случайной матрицы
4. Выход
Введите выбор (1, 2, 3 или 4): 1

Пожалуйста, введите размерность матрицы n (n ≤ 20):
2

Пожалуйста, введите матрицу коэффициентов A (2 строк, 2 столбцов):

Пожалуйста, введите элементы матрицы a11 :
12

Пожалуйста, введите элементы матрицы a12 :
21

Пожалуйста, введите элементы матрицы a21 :
3

Пожалуйста, введите элементы матрицы a22 :
41

Пожалуйста, введите вектор параметров B.

Пожалуйста, введите элементы матрицы b1 :
12

Пожалуйста, введите элементы матрицы b2 :
25
```

```
Исходная матрица:
| 12.0 21.0 | 12.0 |
| 3.0 41.0 | 25.0 |

===== Результаты расчетов =====
Определитель матрицы: det(A) = 429.0

Верхняя треугольная матрица после исключения (расширенная матрица):
| 12.0 21.0 | 12.0 |
| 0.0 35.75 | 22.0 |

Вектор решения x:
x1 = -0.07692307692307694
x2 = 0.6153846153846154

Вектор невязки r = Ax - b:
r1 = 0.0
r2 = 3.552713678800501E-15

Максимальная невязка: 3.552713678800501E-15

Проверка результата (сравнение A*x и b):
Уравнение 1: левая часть = 12.0, правая часть = 12.0, разность = 0.0
Уравнение 2: левая часть = 25.000000000000004, правая часть = 25.0, разность = 3.552713678800501E-15
```

7. Вывод

В результате выполнения данной лабораторной работой я познакомился с различными численными методами решения математических задач на примере систем алгебраических уравнений, реализовав на языке программирования Python метод Гаусса с выбором главного элемента по столбцам. В сравнении с итерационными методами, в данном методе при увеличении порядка системы количество действий стремительно растёт, однако не накапливается погрешность.